SIASA · MANUAL TÉCNICO

Módulo de monitoreo de eventos y alertas

api-SIASA · lectura Modbus del EBO

Arquitectura, funcionamiento interno, configuración, API, base de datos, correo, panel y mantenimiento

Sistema	api-SIASA (Node.js 20, Express 5), contenedor node-api
Módulo	monitoreo/ (eventos_modbus.js, plantilla_correo.js, panel.html) y modbus_cliente/modbus_rtu.js
Sedes con el módulo	IGSS Escuintla (10.0.6.33) · IGSS Villa Nueva (10.0.3.33)
Dirigido a	Equipo técnico y de soporte de SIASA
Fecha	17/10/2026
Versión del documento	1.0
Elaborado por	Soporte técnico SIASA

Documento de uso interno

Control del documento

Versión	Fecha	Descripción	Autor
1.0	17/10/2026	Versión inicial del manual técnico del módulo.	Soporte SIASA

Contenido

1. Introducción 4
1.1 Propósito 4
1.2 Qué hace el módulo 4
1.3 Alcance 4
1.4 Glosario 4
2. Arquitectura 6
2.1 Flujo de datos 6
2.2 Componentes y archivos 6
2.3 Dependencias 7
3. Corrección de la lectura Modbus 8
3.1 El defecto original 8
3.2 La corrección 8
3.3 Prueba de regresión 8
4. Detección de eventos 10
4.1 Puntos de integración 10
4.2 Estado general 10
4.3 Catálogo de eventos 11
4.4 Ciclo de vida de un evento 11
4.5 Evaluación de los medidores 11
4.6 Evaluación periódica (cada 5 s) 12
4.7 Temporizadores del módulo 12
5. Notificaciones por correo 13
5.1 Transporte 13
5.2 Cola, límites y reintentos 13
5.3 Reglas de envío 13
5.4 Plantillas 14
5.5 Correo de prueba y vista previa 15
6. Base de eventos 16
6.1 Aislamiento 16

6.2 Esquema 16
6.3 Escritura y tolerancia a fallas 16
6.4 Retención y volumen 16
6.5 Consultas útiles 17
7. API 18
7.1 Resumen de rutas 18
7.2 Campo lectura en /api/ebo/modbus-auto 18
7.3 GET /estado 19
7.4 GET /eventos 19
7.5 POST /alertas/prueba y GET /alertas/vista-previa 19
8. Panel de monitoreo 20
9. Referencia de configuración 22
9.1 Sede y enlaces 22
9.2 Umbrales 22
9.3 Correo 23
9.4 Base de eventos 23
9.5 Lectura y MQTT 23
10. Registro en el log 25
11. Mantenimiento 26
11.1 Despliegue en una sede 26
11.2 Sedes con el módulo 26
11.3 Diagnóstico 26
11.4 Limitaciones conocidas 27

1. Introducción

1.1 Propósito

Este manual describe el módulo de monitoreo de eventos y alertas de api-SIASA: qué hace, cómo funciona por dentro, cómo se configura, qué expone y cómo se mantiene. Está dirigido al equipo técnico que mantiene, despliega o modifica la API.

El módulo nació del incidente de octubre de 2026 en Escuintla (y del mismo defecto en Villa Nueva): la lectura del EBO quedó bloqueada durante días mientras la API seguía entregando la última lectura al dashboard, sin errores ni avisos. El módulo corrige ese defecto y agrega la vigilancia que faltaba.

1.2 Qué hace el módulo

- **Detecta** cuando los datos dejan de actualizarse, cuando la lectura se atasca, cuando un medidor repite sus valores, sale de línea o queda con voltaje en cero, y cuando hay reconexiones frecuentes al EBO.
- **Notifica** por correo (Microsoft 365) cada evento crítico, agrupa advertencias, envía recordatorios y un correo de resolución, y un **resumen diario** a las 07:00.
- **Registra** cada evento en una base de datos propia (siasa_eventos) y en el log del contenedor con el prefijo [MODBUS-EVENTO].
- **Expone** el estado y el historial por API y en un **panel web**.
- **Se corrige solo**: reconecta si la lectura se atasca y reinicia el proceso si el atasco persiste.
- **Marca la calidad del dato** que recibe el dashboard: hora de la lectura y si está vigente.

1.3 Alcance

El módulo vigila la lectura Modbus automática (versión 1, «modbus-auto») y los medidores ION7400 y PM2130 que esa lectura entrega. No vigila el puente MQTT → BACnet, los sensores LoRaWAN ni el módulo WISE.

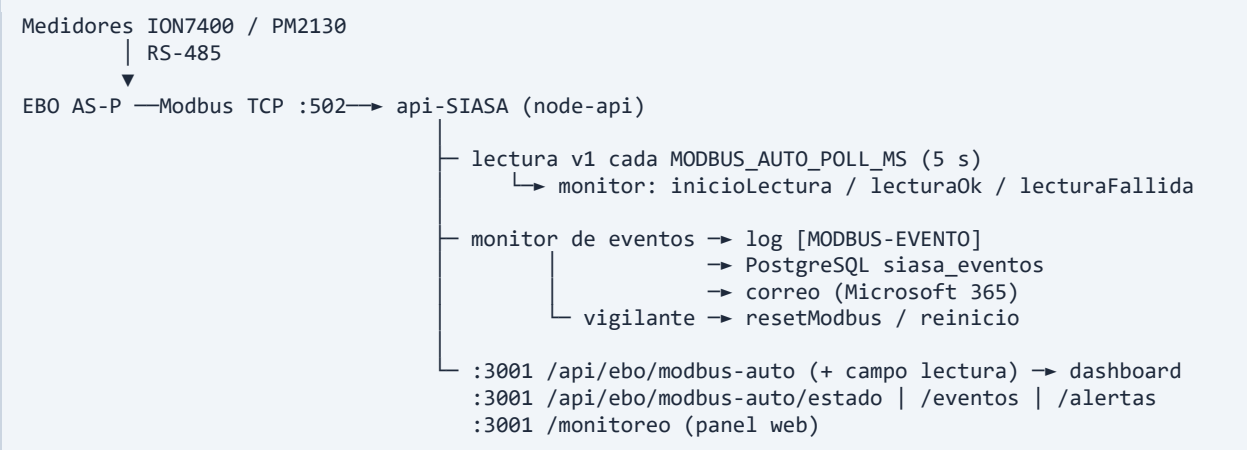
1.4 Glosario

Término	Significado
EBO / AS-P	Controlador EcoStruxure Building Operation que concentra los medidores y los publica por Modbus TCP (puerto 502).
Lectura (snapshot)	Resultado de un barrido completo del EBO. La API guarda en memoria la última lectura buena y la entrega al dashboard.
Evento	Situación anómala con inicio y fin (por ejemplo, datos congelados). Se abre al cumplirse su condición y se cierra al normalizarse.
Evento general	Afecta a la lectura del EBO completa: datos congelados, lectura atascada, reconexiones, reinicios.

Término	Significado
Evento de medidor	Afecta a un medidor: sin cambio, fuera de línea, voltaje en cero.
Vigilante	Lógica que reconecta o reinicia la API cuando la lectura está atascada.
Firma de métricas	Texto con todas las métricas numéricas de un medidor; si no cambia en 15 minutos, el medidor se considera «sin cambio».

2. Arquitectura

2.1 Flujo de datos



2.2 Componentes y archivos

Archivo	Responsabilidad
modbus_cliente/modbus_rtu.js	Conexión Modbus TCP con el EBO, cola de lecturas y reconexión. Contiene la corrección del atasco y el contador de reconexiones.
monitoreo/eventos_modbus.js	Núcleo del módulo: estado en memoria, detección de eventos, apertura y cierre, base de eventos, cola y envío de correos, resumen diario, vigilante y API pública.
monitoreo/plantilla_correo.js	Plantillas HTML de los correos: evento, grupo de eventos y resumen diario.
monitoreo/panel.html	Panel web autocontenido (HTML, CSS y JavaScript sin dependencias externas).
monitoreo/package.json y node_modules/	Dependencia nodemailer 6.10.1, instalada dentro de monitoreo/ para que persista aunque se recree el contenedor.
index.js	Integración: llama al monitor en cada lectura, agrega el campo lectura a la respuesta, define las rutas nuevas e inicia el monitor.
.env	Configuración del módulo (sección 9).
docker-compose.yml	Rotación del log (10 × 50 MB) y PostgreSQL limitado a 127.0.0.1.

2.3 Dependencias

Dependencia	Uso
modbus-serial 8.0.23	Cliente Modbus TCP (ya existente en api-SIASA).
pg	Conexión a PostgreSQL para la base de eventos (ya existente en api-SIASA).
nodemailer 6.10.1	Envío de correo por SMTP. Si no está instalado, el módulo funciona igual y el correo queda desactivado.
PostgreSQL 16 (contenedor postgres-db)	Base de datos siasa_eventos.
Microsoft 365	Servidor de correo smtp.office365.com:587 con STARTTLS.

3. Corrección de la lectura Modbus

3.1 El defecto original

Cuando una lectura terminaba por tiempo de espera o por error de longitud, readRegistersForUnit() y wrapReadWithReconnect() llamaban a resetModbus(), que hacía await client.close(). En modbus-serial 8.0.23, close() solo resuelve su promesa cuando el socket emite «close» estando abierto (openFlag verdadero). Si el EBO ya había cerrado la conexión, el socket estaba destruido, no emitía más eventos y la promesa no se resolvía nunca.

Como la lectura se ejecuta dentro de una cola (withModbusLock), esa promesa pendiente bloqueaba todas las lecturas siguientes. pollModbusAutoSnapshot() quedaba con inFlight en verdadero y no volvía a leer; el endpoint seguía entregando la lectura guardada.

3.2 La corrección

```
async function resetModbus(reason) {
  connected = false;
  connecting = null;
  logReset(reason);                          // [MODBUS-EVENTO] RECONEXION, máx. 1 línea/min
  try {
    await Promise.race([
      Promise.resolve().then(() => client.close()),
      sleep(resetCloseTimeoutMs)              // MODBUS_RESET_CLOSE_TIMEOUT_MS (3000)
    ]);
  } catch (_) {}
  const socket = client._port && client._port._client;
  if (socket && !socket.destroyed) socket.destroy();   // cierre forzado
}
```

- La espera del cierre tiene un límite de 3 segundos; luego el socket se destruye sin depender del EBO.
- Cada reconexión queda en el log con su motivo (por ejemplo, «dir 5620: Data length error»).
- Se exportan resetModbus() (para el vigilante) y getResetTotal() (para contar reconexiones).

3.3 Prueba de regresión

La prueba prueba_reset.js levanta un servidor Modbus simulado que destruye el socket al recibir la primera petición y responde correctamente en las conexiones siguientes. Se ejecuta con:

```
NODE_PATH=<api-SIASA>/node_modules node prueba_reset.js <ruta>/modbus_rtu.js
```

Código	Resultado esperado
Original	**RESULTADO: BLOQUEADO (sin respuesta tras 30 s, conexiones=1)**
Corregido	**RESULTADO: RECUPERADO (conexiones=2, ~5 s)**

4. Detección de eventos

4.1 Puntos de integración

index.js informa al monitor del ciclo de cada lectura automática:

Llamada	Cuándo	Efecto
inicioLectura()	Al empezar pollModbusAutoSnapshot().	Registra la hora de inicio (lectura en curso).
lecturaOk(raw, ms)	Lectura sin error, o lectura directa del endpoint con auto.	Actualiza la última lectura buena, cierra Datos congelados y Lectura atascada, evalúa los medidores.
lecturaFallida(error, ms)	Lectura con error o excepción.	Guarda el último error y las fallas seguidas; escribe LECTURA_FALLIDA (la primera y luego una por minuto).
iniciar(dependencias)	Al arrancar, después de iniciar las lecturas.	Programa los temporizadores, prepara la base y el correo y registra API_REINICIADA.

Dependencias que index.js entrega a iniciar(): obtenerEnCursoDesde() (hora de inicio de la lectura en curso, o null), reconectar(motivo) (resetModbus) y obtenerTotalReconexiones() (getResetTotal).

4.2 Estado general

estadoGeneral() calcula el estado a partir de la antigüedad de la última lectura buena:

Estado	Condición
iniciando	Aún no hay lectura buena y el proceso lleva menos de 60 s.
congelado	Antigüedad ≥ ALERTA_CONGELADO_MIN (5 min).
degradado	Antigüedad ≥ ALERTA_DEGRADADO_S (60 s).
lento	La última lectura tardó ≥ ALERTA_LECTURA_LENTA_S (15 s).
sin_cambio	Estado ok, pero hay al menos un evento Medidor sin cambio abierto.
ok	Ninguna de las anteriores.

4.3 Catálogo de eventos

Evento	Severidad	Se abre	Se cierra
DATOS_CONGELADOS	crítico	Estado congelado (sin lectura buena en 5 min). El inicio es la hora de la última lectura buena.	Con la siguiente lectura buena.
LECTURA_ATASCADA	advertencia	Lectura en curso ≥ 60 s.	Al terminar la lectura.
VIGILANTE_REINICIO	crítico	Lectura en curso ≥ 10 min (si VIGILANTE_REINICIO_ACTIVO). Evento puntual.	Inmediato: el proceso termina.
RECONEXIONES_FRECUENTES	advertencia	Más de 10 reconexiones en la última hora.	Con 5 o menos en la última hora.
MEDIDOR_SIN_CAMBIO	crítico	La firma de métricas no cambia en 15 min.	Al cambiar la firma.
MEDIDOR_FUERA_DE_LINEA	crítico	El EBO reporta el medidor no online durante 3 min.	Al volver a online.
MEDIDOR_EN_CERO	crítico	Todos los voltajes en |v| < 1 V durante 3 min.	Cuando algún voltaje supera 1 V.
API_REINICIADA	info	Al arrancar el monitor. Evento puntual.	Inmediato.

4.4 Ciclo de vida de un evento

Cada evento abierto se guarda en memoria con una clave «EVENTO|dispositivo»; un mismo evento no se abre dos veces mientras siga abierto.

- **abrir()**: crea el código, escribe la línea de log «abierto», inserta la fila en la base y, si es crítico, encola el correo de apertura.
- **cerrar()**: registra la hora de fin, escribe la línea «cerrado», actualiza la fila (fin y duración) y, si el evento fue notificado, encola el correo de resolución.
- **registrarPuntual()**: para eventos sin duración (API_REINICIADA, VIGILANTE_REINICIO): abre y cierra en el mismo momento.

Formato del código de evento: PREFIJO-AAAAMMDD-HHMMSS-xxxx (hora UTC y 4 caracteres hexadecimales aleatorios). Ejemplo: ESC-20261015-145514-2bf9. El prefijo sale de SEDE_PREFIJO (ESC en Escuintla, VNU en Villa Nueva).

4.5 Evaluación de los medidores

evaluarMedidores() se ejecuta con cada lectura buena y recorre raw.modbus_ebo.devices:

- Solo considera identificadores que empiezan con **ion_7400** o **pm_2130**.

- Guarda la ficha del medidor (tipo, modelo, área, edificio, piso, sala, grupo, sistema, capacidad, inventario) y sus métricas, que luego usa el correo.
- **En línea**: status «online», true o «true».
- **Firma de métricas**: todas las métricas con valor numérico, ordenadas por nombre, excepto las que contienen transfer, status o count. Los contadores de energía forman parte de la firma, por lo que un medidor real siempre la cambia.
- **Voltaje en cero**: métricas cuyo nombre contiene volt (excepto unbalance, thd o desbal); se activa si todas valen menos de 1 V en valor absoluto.
- Un medidor fuera de línea no se evalúa para sin cambio ni voltaje en cero.

4.6 Evaluación periódica (cada 5 s)

- Abre DATOS_CONGELADOS si el estado es congelado. La causa probable se elige según el último error **posterior a la última lectura buena**: «Port Not Open», ECONNREFUSED, EHOSTUNREACH, ECONNRESET o EPIPE → sin conexión con el EBO; Timed out o ETIMEDOUT → el EBO no responde Modbus; lectura en curso → lectura atascada.
- Vigila la lectura en curso: a los 60 s abre LECTURA_ATASCADA y pide una reconexión (como máximo una por minuto); a los 10 min registra VIGILANTE_REINICIO, espera 3 s para guardar el evento y enviar el correo, y termina el proceso con código 1. Docker (restart: always) lo vuelve a levantar.
- Actualiza el conteo de reconexiones de la última hora y abre o cierra RECONEXIONES_FRECUENTES.
- Encola recordatorios de los eventos críticos abiertos cada ALERTA_RECORDATORIO_HORAS (4 h).

4.7 Temporizadores del módulo

Tarea	Frecuencia
Evaluación periódica (congelamiento, vigilante, reconexiones, recordatorios)	5 s
Envío de la cola de correos	15 s
Escritura de la cola de la base de datos	15 s
Agrupación de advertencias en un correo	ALERTA_AGRUPAR_ADVERTENCIAS_MIN (15 min)
Comprobación del resumen diario	60 s (se envía una vez al día entre la hora configurada y 10 min después)
Resumen en el log ([MODBUS-EVENTO] RESUMEN)	ALERTA_RESUMEN_LOG_MIN (15 min)
Limpieza de eventos antiguos	Al arrancar y una vez al día, junto con el resumen diario

5. Notificaciones por correo

5.1 Transporte

prepararTransporte() crea el transporte de nodemailer solo si se cumplen tres condiciones; si no, el correo queda desactivado y el motivo aparece en /estado:

- ALERTAS_CORREO_ACTIVO=true;
- SMTP_HOST y ALERTAS_CORREO_PARA definidos;
- nodemailer instalado (se busca primero en monitoreo/node_modules).

Tiempos de espera del transporte: conexión 15 s, saludo 15 s, socket 30 s.

5.2 Cola, límites y reintentos

- Todos los correos pasan por una cola en memoria que se procesa cada 15 s, en orden.
- Límite de ALERTA_MAX_CORREOS_POR_HORA (20) por hora; al alcanzarlo se escribe LIMITE_CORREOS y el resto espera.
- Si un envío falla se escribe CORREO_FALLIDO y se reintenta en el siguiente ciclo; tras 50 intentos el correo se descarta.
- Cada envío exitoso escribe CORREO_ENVIADO y actualiza notificado_en o cierre_notificado_en en la base.

5.3 Reglas de envío

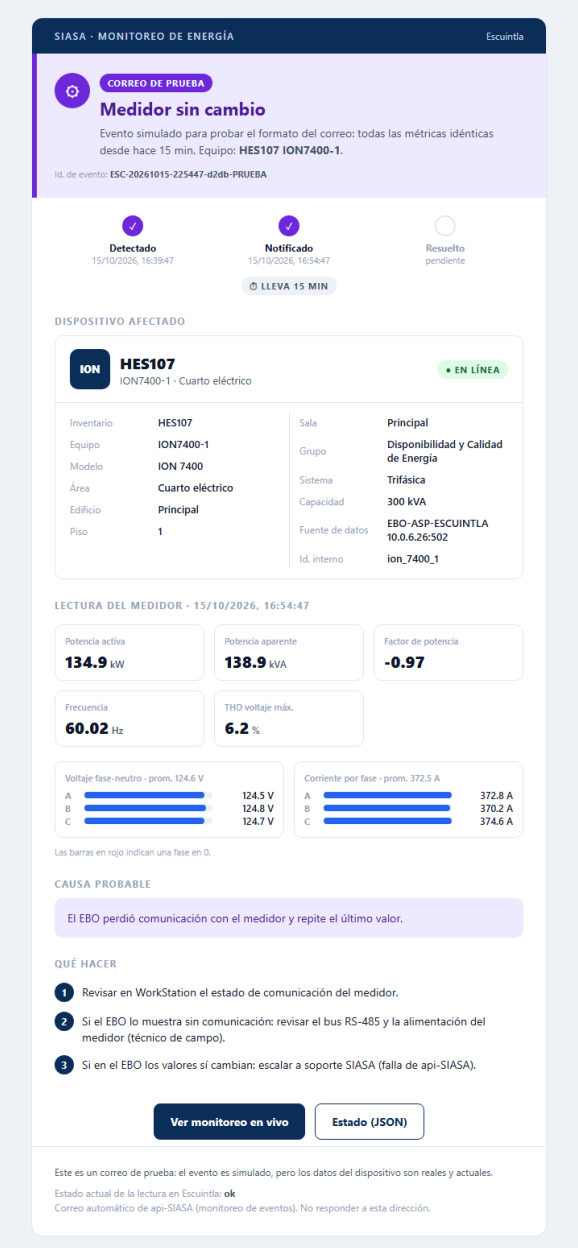
Situación	Correo
Evento crítico general	Inmediato.
Evento crítico de 1 o 2 medidores en la misma evaluación	Un correo por medidor.
Eventos de 3 o más medidores en la misma evaluación	Un solo correo agrupado («posible falla común del EBO»).
Advertencias	Agrupadas cada 15 min; si es una sola, se envía con el formato de evento.
Evento crítico abierto	Recordatorio cada 4 h.
Cierre de un evento notificado	Correo de resolución (agrupado si son 3 o más medidores).
Resumen diario	Una vez al día a ALERTA_RESUMEN_DIARIO_HORA (07:00, zona ALERTAS_ZONA_HORARIA).

5.4 Plantillas

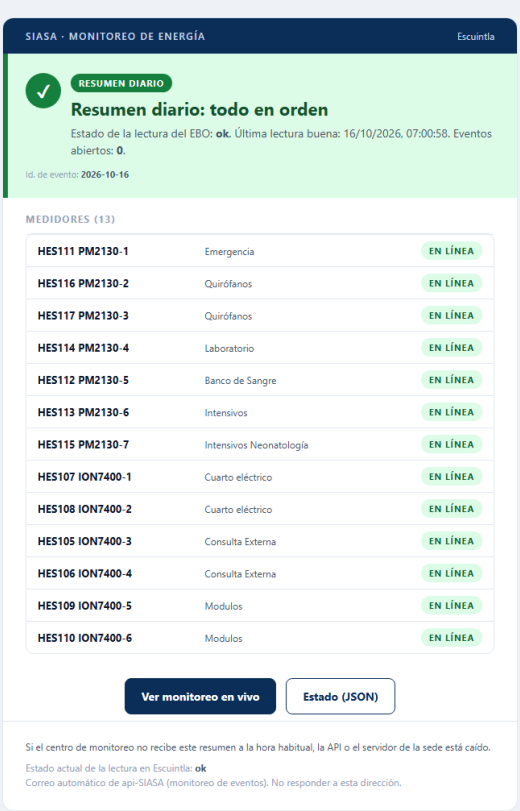
Función	Contenido
correoEvento()	Encabezado por severidad, línea de tiempo (detectado, notificado, resuelto), ficha del dispositivo, lectura del medidor con barras por fase, causa probable, pasos a seguir y botones. En eventos generales muestra la tarjeta de lectura del EBO.
correoGrupo()	Lista de medidores afectados con su evento y hora, pasos a seguir y botones.
correoResumen()	Estado de la lectura, eventos abiertos y la lista de medidores con su estado.

Las plantillas usan tablas y estilos en línea para verse correctamente en Outlook y Microsoft 365. Cada correo incluye además una versión en texto plano. El HTML se arma en el momento del evento, con los valores de ese instante.

Colores de encabezado: rojo (crítico), ámbar (advertencia), azul (informativo), verde (resuelto) y violeta (prueba).



(a) *correoEvento(): alerta de prueba con datos reales (Escuintla, HES107).*



(b) *correoResumen(): resumen diario (Escuintla, 13 medidores).*

Figura 1. *Correos enviados por el sistema, con los datos reales de la sede.*

5.5 Correo de prueba y vista previa

eventoDePrueba() arma un evento simulado MEDIDOR_SIN_CAMBIO (u otro evento de medidor indicado) con los datos reales y actuales de un medidor. Por defecto usa el inventario HES107; si no existe, el primer medidor de la sede.

- **enviarCorreoPrueba()** lo envía por SMTP con el asunto [PRUEBA] y devuelve el identificador del mensaje.
- **vistaPreviaCorreo()** devuelve el HTML sin enviarlo.

6. Base de eventos

6.1 Aislamiento

La base siasa_eventos vive en el mismo contenedor postgres-db, pero con un usuario propio (siasa_eventos) que no tiene acceso a mi_basedatos. El módulo usa su propio pool de conexiones (máximo 2), independiente de auth/db.js.

```
CREATE USER siasa_eventos WITH PASSWORD '<aleatoria de 32 caracteres>';
CREATE DATABASE siasa_eventos OWNER siasa_eventos;
REVOKE CONNECT ON DATABASE mi_basedatos FROM PUBLIC;
GRANT CONNECT ON DATABASE mi_basedatos TO admin;
```

6.2 Esquema

La tabla y sus índices se crean automáticamente al arrancar (prepararTabla):

```
CREATE TABLE IF NOT EXISTS api_eventos (
  id              BIGSERIAL PRIMARY KEY,
  codigo          TEXT UNIQUE NOT NULL,
  sede            TEXT NOT NULL,
  evento          TEXT NOT NULL,
  severidad       TEXT NOT NULL,          -- critico | advertencia | info
  dispositivo     TEXT,
  inventario      TEXT,
  inicio          TIMESTAMPTZ NOT NULL,
  fin             TIMESTAMPTZ,            -- NULL = abierto
  duracion_s      INTEGER,
  detalle         JSONB,                  -- mensaje, causa, area, ultimo_error...
  notificado_en   TIMESTAMPTZ,
  cierre_notificado_en TIMESTAMPTZ,
  intentos_correo SMALLINT NOT NULL DEFAULT 0,
  creado_en       TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS api_eventos_inicio_idx   ON api_eventos (inicio DESC);
CREATE INDEX IF NOT EXISTS api_eventos_abiertos_idx ON api_eventos (fin) WHERE fin IS NULL;
```

6.3 Escritura y tolerancia a fallas

- Todas las escrituras pasan por una cola en memoria (máximo 5000 operaciones; si se llena, se descartan las más antiguas) que se vacía cada 15 s en orden.
- Si la base no responde, las operaciones esperan en la cola y se reintentan; el error se escribe en el log como máximo una vez cada 10 min. La lectura y los correos no se detienen.
- Si EVENTOS_DB_HOST o EVENTOS_DB_PASSWORD están vacíos, la base queda desactivada y los eventos solo se registran en el log y en memoria.

6.4 Retención y volumen

limpiarAntiguos() elimina los eventos cerrados cuyo inicio tenga más de EVENTOS_RETENCION_MESES (12). Cada evento es una fila, no una lectura: el volumen esperado es de pocos MB por sede al año.

6.5 Consultas útiles

```
-- Eventos abiertos
SELECT codigo, evento, dispositivo, inicio FROM api_eventos WHERE fin IS NULL;

-- Resumen de los últimos 7 días
SELECT evento, severidad, count(*) AS veces, sum(duracion_s)/60 AS minutos
  FROM api_eventos WHERE inicio > now() - interval '7 days'
 GROUP BY evento, severidad ORDER BY veces DESC;

-- Acceso desde el servidor
docker exec -it postgres-db psql -U admin -d siasa_eventos
```

7. API

Todas las rutas están en el puerto 3001 (modbusAutoApp) y requieren la clave de la API en el header x-api-key (o en el parámetro api_key), salvo la página /monitoreo.

7.1 Resumen de rutas

Método y ruta	Descripción
GET /api/ebo/modbus-auto	Datos de los medidores (ya existente). Ahora incluye el campo lectura.
GET /api/ebo/modbus-auto/estado	Estado general, medidores y eventos abiertos.
GET /api/ebo/modbus-auto/eventos	Historial de eventos desde la base.
POST /api/ebo/modbus-auto/alertas/prueba	Envía un correo de prueba con datos reales.
GET /api/ebo/modbus-auto/alertas/vista-previa	Devuelve el HTML del correo de prueba sin enviarlo.
GET /monitoreo	Panel web (no requiere clave para cargar; la pide para consultar datos).

7.2 Campo lectura en /api/ebo/modbus-auto

Cuando la respuesta sale de la lectura guardada, se agrega:

```
"lectura": {
  "scanned_at": "2026-10-17T04:47:53.944Z",   // hora de la última lectura buena
  "antiguedad_s": 5,
  "estado": "ok",                              // ver 4.2
  "stale": false                              // true si degradado o congelado
}
```

Con ALERTA_RESPONDER_503=true y estado congelado, la ruta responde 503 con { message, error, lectura } en lugar de entregar datos viejos. Por defecto está desactivado para no afectar al dashboard.

7.3 GET /estado

```
{
  "sede": "Escuintla", "estado": "ok", "motivo": null,
  "ultima_lectura_ok": "...", "antiguedad_s": 3,
  "ultimo_intento": "...", "ultimo_error": null, "ultimo_error_en": null,
  "fallas_consecutivas": 0, "lectura_en_curso_desde": "...",
  "duracion_ultima_lectura_ms": 4730, "reconexiones_ultima_hora": 0,
  "eventos_abiertos": [ { "codigo", "evento", "severidad", "dispositivo", "inventario", "desde"
} ],
  "dispositivos": [ { "id", "nombre", "inventario", "area", "estado", "ultimo_cambio", "evento"
} ],
  "base_de_eventos": "ok", "pendientes_base": 0,
  "correo": "activo", "pendientes_correo": 0,
  "proceso_iniciado": "..."
}
```

Estados de dispositivo: ok, fuera_de_linea, sin_cambio, en_cero. Si el correo está desactivado, el campo correo indica el motivo.

7.4 GET /eventos

Parámetro	Descripción
desde, hasta	Fecha u hora ISO (filtra por inicio).
dispositivo	Nombre o inventario del medidor.
abiertos=true	Solo eventos sin fin.
limite	Máximo de filas (1 a 2000, por defecto 200).

Respuesta: { origen: «base de datos», total, eventos: [...] } ordenada por inicio descendente. Si la base está desactivada devuelve los eventos abiertos en memoria; si no responde, 503.

7.5 POST /alertas/prueba y GET /alertas/vista-previa

Parámetros opcionales: dispositivo (nombre o inventario) y evento (MEDIDOR_SIN_CAMBIO, MEDIDOR_FUERA_DE_LINEA o MEDIDOR_EN_CERO).

```
K=$(grep ^MODBUS_AUTO_API_KEY= ~/node-postgres-docker/api/.env | cut -d= -f2-)
curl -s -X POST -H "x-api-key: $K" \
  "http://localhost:3001/api/ebo/modbus-auto/alertas/prueba?dispositivo=HES107"

{"enviado":true,"para":"...","asunto":"[PRUEBA] SIASA Escuintla - HES107 ION7400-1 - Medidor sin
cambio",
 "dispositivo":"ION7400-1","id_mensaje":"<...@siasa.gt>"}
```

Si el correo está desactivado responde { enviado: false, motivo }; si el servidor SMTP rechaza el envío, 502.

8. Panel de monitoreo

monitoreo/panel.html es una página autocontenida que se sirve en http://<servidor>:3001/monitoreo. No usa librerías externas, por lo que funciona en redes sin internet.

- **Acceso**: pide la clave de la API la primera vez y la guarda en el navegador (localStorage). También acepta ?key= en la URL.
- **Datos**: consulta /estado y /eventos cada 15 s.
- **Contenido**: estado de la lectura, antigüedad del dato, eventos abiertos, medidores en línea, reconexiones, estado del correo, tarjeta por medidor, gráfico de eventos por día, historial con filtros y panel lateral con el detalle de cada evento.
- **Acciones**: vista previa del correo y envío del correo de prueba (solo con el correo activo).
- **Tema** claro u oscuro, según el sistema o el botón del encabezado.

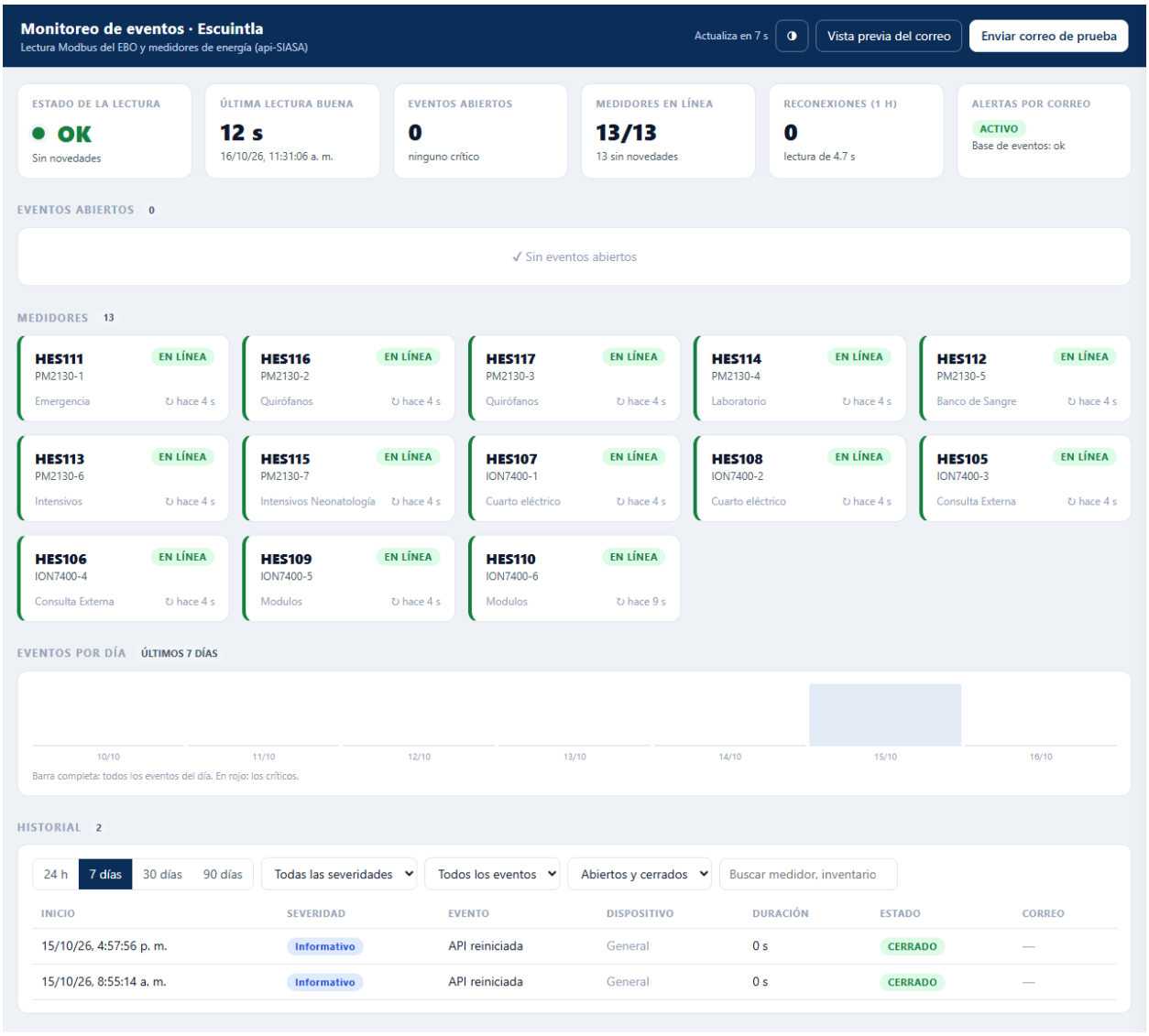


Figura 2. *Panel de monitoreo (Escuintla).*

9. Referencia de configuración

Todas las variables van en api/.env. Los cambios se aplican con docker restart node-api. Los valores entre paréntesis son los predeterminados.

9.1 Sede y enlaces

Variable	Predeterminado	Descripción
SEDE_NOMBRE	Sede	Nombre en asuntos, correos y panel.
SEDE_PREFIJO	3 letras de SEDE_NOMBRE	Prefijo del código de evento (ESC, VNU...).
ALERTAS_ZONA_HORARIA	America/Guatemala	Zona para las horas de los correos y del resumen diario.
ALERTAS_URL_ESTADO	(vacío)	Enlace al estado en los correos.
ALERTAS_URL_MONITOREO	derivado de URL_ESTADO	Enlace al panel en los correos.

9.2 Umbrales

Variable	Predeterminado	Descripción
ALERTA_CONGELADO_MIN	5	Minutos sin lectura buena para Datos congelados.
ALERTA_DEGRADADO_S	60	Segundos sin lectura buena para el estado degradado.
ALERTA_LECTURA_LENTA_S	15	Segundos de duración para el estado lento.
ALERTA_ATASCADA_S	60	Segundos de lectura en curso para Lectura atascada.
VIGILANTE_REINICIO_ACTIVO	true	Permite al vigilante reiniciar el proceso.
VIGILANTE_REINICIO_MIN	10	Minutos de atasco para reiniciar.
ALERTA_SIN_CAMBIO_MIN	15	Minutos con la misma firma para Medidor sin cambio.
ALERTA_FUERA_DE_LINEA_MIN	3	Minutos fuera de línea para alertar.
ALERTA_EN_CERO_MIN	3	Minutos con voltaje en 0 para alertar.
ALERTA_RECONEXIONES_POR_HORA	10	Reconexiones por hora para Reconexiones frecuentes.
ALERTA_RESPONDER_503	false	Responder 503 en lugar de datos congelados.

9.3 Correo

Variable	Predeterminado	Descripción
ALERTAS_CORREO_ACTIVO	false	Activa el envío de correos.
SMTP_HOST	(vacío)	smtp.office365.com en las sedes actuales.
SMTP_PORT	587	Puerto SMTP.
SMTP_SEGURO	false	true para TLS directo (465); false usa STARTTLS.
SMTP_USUARIO / SMTP_CLAVE	(vacío)	Credenciales. La clave va entre comillas simples si tiene caracteres especiales.
ALERTAS_CORREO_DE	SMTP_USUARIO	Remitente, por ejemplo «Alarmas SIASA <alarmas.atlas@siasa.gt>».
ALERTAS_CORREO_PARA / _CC	(vacío)	Destinatarios separados por coma.
ALERTA_RECORDATORIO_HORAS	4	Horas entre recordatorios.
ALERTA_AGRUPAR_ADVERTENCIAS_MIN	15	Minutos para agrupar advertencias.
ALERTA_MAX_CORREOS_POR_HORA	20	Límite de correos por hora.
ALERTA_RESUMEN_DIARIO_HORA	07:00	Hora del resumen diario (HH:MM).
ALERTA_RESUMEN_LOG_MIN	15	Minutos entre resúmenes en el log.

9.4 Base de eventos

Variable	Predeterminado	Descripción
EVENTOS_DB_HOST	(vacío = desactivada)	postgres (nombre del servicio en el compose).
EVENTOS_DB_PORT	5432	Puerto.
EVENTOS_DB_NAME / _USER	siasa_eventos	Base y usuario.
EVENTOS_DB_PASSWORD	(vacío = desactivada)	Clave del usuario siasa_eventos.
EVENTOS_RETENCION_MESES	12	Meses que se conservan los eventos cerrados.

9.5 Lectura y MQTT

Variable	Valor en las sedes	Descripción
MODBUS_RESET_CLOSE_TIMEOUT_MS	3000 (predeterminado)	Espera máxima del cierre de conexión antes de destruir el socket.

Variable	Valor en las sedes	Descripción
MODBUS_AUTO_V2_POLL_MS	0	0 desactiva el lector v2 en segundo plano (no se usa).
MQTT_ENABLED	false	Conexión MQTT de la API.
MQTT_SENSOR_DB_ENABLED	false	Segundo candado: solo con ambas en true se insertan métricas en sensores_mqtt.

10. Registro en el log

El módulo escribe una línea JSON por evento con el prefijo [MODBUS-EVENTO]. Para verlas:

```
docker logs --since 24h node-api 2>&1 | grep MODBUS-EVENTO
```

evento	Cuándo
MONITOREO_INICIADO	Al arrancar: base, correo y umbrales en uso.
<EVENTO> con accion «abierto» / «cerrado»	Apertura y cierre de cada evento del catálogo, con su código.
RECONEXION	Reconexión al EBO con el motivo (máximo una línea por minuto, con conteo).
LECTURA_FALLIDA	Primera falla y luego una por minuto mientras siga fallando.
LECTURA_LENTA	Lectura de 15 s o más (máximo una por minuto).
VIGILANTE_RECONEXION	El vigilante pidió reconectar.
RESUMEN	Cada 15 min: estado, lecturas buenas y fallidas, duración promedio, reconexiones, pendientes.
CORREO_ENVIADO / CORREO_FALLIDO	Resultado de cada envío.
LIMITE_CORREOS	Se alcanzó el límite de correos por hora.
CORREO_PRUEBA	Se envió un correo de prueba.

Además se escriben mensajes de la base de eventos: «base de eventos lista», errores de conexión y la cantidad de eventos eliminados por la limpieza.

11. Mantenimiento

11.1 Despliegue en una sede

El código de cada sede puede ser distinto, por lo que el módulo se integra sobre el index.js propio de la sede, no copiando el de otra. Pasos:

1. Respaldar docker-compose.yml, api/.env, api/index.js y api/modbus_cliente/modbus_rtu.js, y extraer el log del contenedor.
2. Reemplazar modbus_rtu.js por la versión corregida (ajustando la IP del EBO por defecto) y copiar la carpeta monitoreo/.
3. Aplicar en index.js los puntos de integración de la sección 4.1, el campo lectura (7.2), las rutas (7.1) y el doble candado de MQTT.
4. Agregar en docker-compose.yml la rotación del log y limitar el puerto de PostgreSQL a 127.0.0.1.
5. Encender PostgreSQL con docker-compose up -d --no-deps postgres y verificar que no hay inserciones; crear la base siasa_eventos (6.1).
6. Agregar las variables de la sección 9 en api/.env e instalar nodemailer: docker exec node-api npm install --prefix /app/monitoreo nodemailer@6.10.1.
7. Recrear node-api: docker-compose rm -sf api y docker-compose up -d --no-deps api (evita el error KeyError: 'ContainerConfig' de docker-compose 1.29 con Docker 28).
8. Verificar /estado, /eventos, la respuesta de /api/ebo/modbus-auto, el panel, y enviar el correo de prueba.

11.2 Sedes con el módulo

Sede	Servidor	EBO	Medidores	Prefijo	Instalado
Escuintla	10.0.6.33	10.0.6.26	13 (6 ION7400, 7 PM2130)	ESC	15/10/2026
Villa Nueva	10.0.3.33	10.0.3.26	7 (1 ION7400, 6 PM2130)	VNU	16/10/2026

11.3 Diagnóstico

Síntoma	Revisar	Solución
/estado indica correo «desactivado: ...»	El motivo en el mismo campo.	Completar SMTP_HOST / ALERTAS_CORREO_PARA, poner ALERTAS_CORREO_ACTIVO=true o instalar nodemailer.
CORREO_FALLIDO con «535 Authentication»	Usuario y clave; SMTP AUTH habilitado en el buzón de Microsoft 365.	Corregir la clave o pedir que se habilite SMTP AUTH.

Síntoma	Revisar	Solución
CORREO_FALLIDO con timeout	Salida del servidor al puerto 587.	Pedir a Redes que permita smtp.office365.com:587.
/estado indica base «sin conexion»	docker ps (postgres-db) y EVENTOS_DB_*.	Levantar postgres-db; los eventos esperan en cola.
Muchas líneas RECONEXION	El motivo de la línea (registro y error).	Si viene de un lector no usado, apagarlo; si es la red, escalar a Redes.
Medidor sin cambio que no es real	Si las métricas del medidor son siempre iguales (medidor sin carga).	Revisar el medidor; si es un caso normal, ajustar ALERTA_SIN_CAMBIO_MIN.
No llega el resumen diario	Que la API esté arriba y ALERTA_RESUMEN_DIARIO_HORA.	Si la API está caída, revisar el servidor.

11.4 Limitaciones conocidas

- Si el servidor o el contenedor se detienen, el módulo no puede enviar alertas. Lo cubre la ausencia del resumen diario; se recomienda un vigilante externo.
- El estado de los eventos abiertos vive en memoria: tras un reinicio, un evento que seguía abierto en la base queda sin cerrar en ella, y si la condición persiste se abre uno nuevo.
- El módulo vigila la lectura v1. Si una sede usa la v2 como principal, hay que conectar el monitor a refreshModbusAutoV2Snapshot().
- La clave de la API del panel se guarda en el navegador del usuario.